JOBSHEET 5

Brute Force dan Divide Conquer

Nama : Rachmah Nur Chotimah

NIM : 254107060052

Kelas : SIB 1A

No.Absen : 20

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
   Jawab :
   - **Bagian if (n == 1)**: Berfungsi sebagai **Base Case** (kondisi dasar), yaitu titik di mana perulangan rekursif harus berhenti dan langsung mengembalikan nilai 1.
   - **Bagian else**: Berfungsi sebagai **Recursive Step**, di mana masalah utama dipecah menjadi sub-masalah yang lebih kecil dengan cara memanggil kembali *method* faktorialDC menggunakan parameter n-1.
2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
   Jawab :
   Sangat memungkinkan. Perulangan for pada *Brute Force* bisa diganti menggunakan struktur perulangan while atau do-while.
   - Bukti :

```java
public class Faktorial {
    int faktorialBF(int n) {
        int fakto = 1;
        int i = 1;
        while (i <= n) {
            fakto = fakto * i;
            i++;
        }
        return fakto;
    }
```

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
   Jawab :
   - fakto *= i; : Merupakan pendekatan Iteratif (*Brute Force*), di mana nilai variabel fakto dikalikan secara langsung dengan nilai pencacah i di dalam sebuah *loop* yang sudah ada .
   - int fakto = n * faktorialDC(n-1); : Merupakan pendekatan Rekursif (*Divide Conquer*), di mana program tidak menggunakan *looping* manual, melainkan memanggil dirinya sendiri untuk mendapatkan hasil dari sub-masalah (n-1) sebelum dikalikan dengan n.
4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!
   Jawab :
   - faktorialBF(): Bekerja secara linier dan sekuensial menggunakan perulangan untuk menyelesaikan masalah dari awal sampai akhir dalam satu alur eksekusi.
   - faktorialDC(): Bekerja dengan prinsip membagi masalah menjadi bagian-bagian kecil yang serupa (rekursif) sampai mencapai kondisi paling

sederhana (*base case*), lalu menggabungkan hasilnya kembali menjadi solusi akhir.

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

5.3.3. Pertanyaan

1) Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
   Jawab :
   - pangkatBF(): Menggunakan algoritma *Brute Force* dengan perulangan for sederhana yang melakukan perkalian sebanyak n kali secara linier. Kompleksitas waktunya adalah O(n).
   - pangkatDC(): Menggunakan algoritma *Divide and Conquer* yang membagi pangkat menjadi dua (n/2) di setiap tahap rekursif, sehingga proses perkalian jauh lebih sedikit dan efisien. Kompleksitas waktunya adalah O(log n).
2) Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!
   Jawab :
   Sudah. Tahap *combine* terjadi pada baris kode yang melakukan operasi perkalian hasil rekursi.

```
int pangkatDC(int a, int n) {
    if (n == 1) {
        return a;
    } else {
        if(n%2==1){
            return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
        } else {
            return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
        }
    }
}
```

3) Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
   Jawab :
   Parameter membuat method tersebut lebih fleksibel untuk menghitung angka lain tanpa harus mengubah atribut objek.

```
int pangkatBF(int a,int n) {
    int hasil = 1;
    for (int i = 0; i <= pangkat; i++) {
        hasil *= a;
    }
    return hasil;
}
```

4) Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!
   Jawab :
   - pangkatBF(): Melakukan iterasi sebanyak n kali untuk mengalikan basis secara berulang sampai mencapai jumlah pangkat yang diinginkan.
   - pangkatDC(): Memecah masalah pangkat n menjadi n/2 secara terus-menerus (divide), menyelesaikannya secara rekursif (conquer), lalu mengalikan hasilnya kembali (combine) dengan mempertimbangkan apakah n bernilai ganjil atau genap

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

5.4.3. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
   Jawab :
   Variabel mid digunakan untuk menentukan titik tengah atau pembagi dari rentang array yang sedang diproses. Variabel ini menjadi acuan untuk membagi persoalan (*divide*) menjadi dua bagian, yaitu bagian kiri (dari indeks l sampai mid) dan bagian kanan (dari mid+1 sampai r).
2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?
   ```
   double lsum = totalDC(arr, l, mid);
   double rsum = totalDC(arr, mid+1, r);
   ```
   Jawab :
   - double lsum = totalDC(arr, l, mid);: Statement ini melakukan pemanggilan rekursif untuk menyelesaikan (*conquer*) penjumlahan elemen pada bagian kiri array.
   - double rsum = totalDC(arr, mid+1, r);: Statement ini melakukan pemanggilan rekursif untuk menyelesaikan (*conquer*) penjumlahan elemen pada bagian kanan array.
3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?
   ```
   return lsum+rsum;
   ```
   Jawab :
   Penjumlahan ini merupakan tahap **combine**, yaitu menggabungkan hasil solusi dari sub-masalah bagian kiri dan sub-masalah bagian kanan untuk mendapatkan total nilai keseluruhan dari masalah semula.
4. Apakah base case dari totalDC()?
   Jawab :
   Base case dari method ini adalah saat kondisi if (l == r) terpenuhi. Kondisi ini terjadi ketika sub-array hanya menyisakan satu elemen saja, sehingga method langsung mengembalikan nilai elemen tersebut (arr[l]) tanpa melakukan pembelahan lagi.
5. Tarik Kesimpulan tentang cara kerja totalDC()
   Jawab :
   totalDC() bekerja dengan membagi array secara terus-menerus menjadi dua bagian hingga tiap bagian hanya memiliki satu elemen (*divide*). Setiap elemen

tunggal tersebut dianggap sebagai solusi dasar yang kemudian dijumlahkan secara bertahap saat alur rekursif kembali ke atas (*conquer* dan *combine*) sampai seluruh elemen terakumulasi menjadi total keuntungan.

4.5 Latihan Praktikum

1. Sebuah kampus memiliki daftar nilai mahasiswa dengan data sesuai tabel di bawah ini

Nama	NIM	Tahun Masuk	Nilai UTS	Nilai UAS
Ahmad	220101001	2022	78	82
Budi	220101002	2022	85	88
Cindy	220101003	2021	90	87
Dian	220101004	2021	76	79
Eko	220101005	2023	92	95
Fajar	220101006	2020	88	85
Gina	220101007	2023	80	83
Hadi	220101008	2020	82	84

Tentukan:

a) Nilai UTS tertinggi menggunakan **Divide and Conquer**!

b) Nilai UTS terendah menggunakan **Divide and Conquer**!

c) Rata-rata nilai UAS dari semua mahasiswa menggunakan **Brute Force**!

Jawab :

a) Mencari Nilai UTS Tertinggi dengan DC

```
int cariMaxUTS(Mahasiswa arr[], int l, int r) {
    if (l == r) {
        return arr[l].nilaiUTS;
    }
    int mid = (l + r) / 2;
    int maxKiri = cariMaxUTS(arr, l, mid);
    int maxKanan = cariMaxUTS(arr, mid + 1, r);
    return Math.max(maxKiri, maxKanan);
}
```

b) Mencari Nilai UTS Terendah dengan DC

```
int cariMinUTS(Mahasiswa arr[], int l, int r) {
    if (l == r) {
        return arr[l].nilaiUTS;
    }
    int mid = (l + r) / 2;
    int minKiri = cariMinUTS(arr, l, mid);
    int minKanan = cariMinUTS(arr, mid + 1, r);
    return Math.min(minKiri, minKanan);
}
```

c) Mencari Rata-Rata nilai UAS semua nilai mahasiswa menggunakan BF

```
double hitungRataRataUAS(Mahasiswa[] mhs) {
    double total = 0;
    for (int i = 0; i < mhs.length; i++) {
        total += mhs[i].nilaiUAS;
    }
    return total / mhs.length;
}
```

LINK GITHUB : https://github.com/lovvovala/https---github.com-lovvovala-PASD/tree/main/Jobsheet5/BFdanDC